A Project Report on

AI POWERED AUTOMATIC NEWS SUMMARIZATION SYSTEM

Using DistilBART and Natural Language Processing: A Comparative Study with TextRank

Submitted in partial fulfilment of the requirements of the Elective Project

Bachelor of Technology in Electronics & Telecommunication Engineering

Submitted by

Srishti Chowdhury – Roll No. 2304125

Umang Gupta – Roll No. 2304133

Tushar Ranjan Dash – Roll No. 2304132

Abir Das – Roll No. 2304138

SUB- MLLP

7TH SEMESTER

Academic Year 2026–2027

KIIT DEEMED TO BE UNIVERSITY

School of Electronics Engineering, Bhubaneswar, Odisha

CERTIFICATE

This is to certify that the project report entitled **"AI Powered Automatic News Summarization System using DistilBART and Natural Language Processing"** is a bonafide record of the work carried out by

Srishti Chowdhury (Roll No. 2304125)

Umang Gupta (Roll No. 2304133)

Tushar Ranjan Dash (Roll No. 2304132)

Abir Das (Roll No. 2304138)

in partial fulfilment of the requirements of the Elective Project in the Bachelor of Technology programme in Computer Science and Engineering at KIIT Deemed to be University, Bhubaneswar, during the academic year 2026–2027.

The work embodied in this report has not been submitted to any other university or institute for the award of any degree or diploma.

Date: _____ Place: Bhubaneswar

DECLARATION

We hereby declare that the project report entitled **"AI Powered Automatic News Summarization System using DistilBART and Natural Language Processing"** submitted to KIIT Deemed to be University is an original record of work done by us. The pretrained model checkpoint (sshleifer/distilbart-cnn-12-6) and open-source libraries used in the project are credited in the References, and any text taken from published sources has been acknowledged.

We further declare that the contents of this report have not been submitted, in part or in full, for any other degree or diploma. Wherever evaluation results are reported, they are those produced by running the evaluation scripts described in the report on the stated test set.

Srishti Chowdhury (2304125) Signature: _____

Umang Gupta (2304133) Signature: _____

Tushar Ranjan Dash (2304132) Signature: _____

Abir Das (2304138) Signature: _____

Date: _____ Place: Bhubaneswar

ACKNOWLEDGEMENT

We express our sincere gratitude to our project guide, Suprava Pattnaik, School of Electronics Engineering, KIIT Deemed to be University, for the guidance, encouragement and constructive feedback that shaped this project from its first proposal to its final report.

We thank the Head of the School of Electronics Engineering and the faculty members for providing the academic environment and laboratory facilities needed to carry out this work. We are also grateful to the developers and maintainers of the open-source tools on which the project depends, namely Python, PyTorch, Hugging Face Transformers, Trafilatura, scikit-learn, NetworkX and Streamlit, and to the authors of the BART, DistilBART and TextRank research papers whose ideas we studied and applied.

Finally, we thank our families and friends for their constant support during the project.

Srishti Chowdhury (2304125)
Umang Gupta (2304133)
Tushar Ranjan Dash (2304132)
Abir Das (2304138)

ABSTRACT

Readers face a continuous stream of online news, and most articles contain background, repetition and secondary events that hide the central development. Automatic text summarization addresses this by producing a short version of an article that keeps the facts that matter. This project builds an AI-powered news summarization system and studies two competing approaches to the task.

The abstractive path uses DistilBART (sshleifer/distilbart-cnn-12-6), a distilled sequence-to-sequence Transformer fine-tuned on CNN/DailyMail. The system accepts a news URL, extracts the article body and metadata with Trafilatura, cleans and normalises the text, tokenises it with the Hugging Face tokenizer, and splits long articles into chunks that fit the 1,024-token input limit. Each chunk is summarised with beam search (beam width 2, no repeated 3-grams), and a second pass condenses the partial summaries when the article is long. The user selects a short (20–80 tokens), medium (40–130 tokens) or long (60–180 tokens) summary. The system reports word counts, the compression ratio and a timestamp, and saves a TXT report.

The extractive path is a TextRank baseline: sentences are represented as TF-IDF vectors, connected in a cosine-similarity graph, ranked with PageRank, and the top sentences are returned in source order. The two methods are compared on ROUGE-1, ROUGE-2 and ROUGE-L, on sentence-level factual consistency measured with a natural-language-inference model, and on latency. An evaluation script (evaluate.py) that runs the comparison on a CNN/DailyMail test sample is provided with the report.

The report documents the design, implementation and evaluation protocol. The numerical comparison cells in Chapter 5 are marked as pending until the evaluation script has been run on the chosen test set, so that no figure is reported that was not measured. The report also discusses why ROUGE overlap alone cannot establish factual accuracy, and why extractive copying lowers the risk of invented content without removing the risk of lost context.

Keywords: *Text summarization, TextRank, DistilBART, BART, Transformer, ROUGE, factual consistency, NLP, news.*

TABLE OF CONTENTS

CHAPTER 1 – INTRODUCTION 1
1.1 Introduction to Natural Language Processing (NLP) 1
1.2 Background of Automatic Text Summarization 1
1.3 Need for News Summarization 1
1.4 Problem Statement 2
1.5 Objectives of the Project 2
1.6 Scope of the Project 2
1.7 Existing System and Its Limitations 2
1.8 Proposed System 3
1.9 Organization of the Report 3
CHAPTER 2 – LITERATURE REVIEW AND TECHNOLOGY OVERVIEW 4
2.1 Introduction to Text Summarization Techniques 4
2.2 Extractive vs Abstractive Text Summarization 4
2.3 Introduction to Transformer Architecture 4
2.4 Overview of BART Model 5
2.5 DistilBART and Its Working Principle 5
2.6 Role of Natural Language Processing in the Project 5
2.7 Technologies and Tools Used 6
2.8 Hardware and Software Requirements 6
2.9 TextRank for Extractive Summarization 7
2.10 Evaluation Metrics: ROUGE and Factual Consistency 7
CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN 8
3.1 System Architecture 8
3.2 Overall Workflow of the Proposed System 8
3.3 Functional Requirements 9
3.4 Non-Functional Requirements 9
3.5 Module Description 9
3.6 Data Flow Diagram (DFD) 10
3.7 Use Case Diagram 10
3.8 Flowchart of the Proposed System 12
3.9 Input and Output Design 12
CHAPTER 4 – METHODOLOGY AND IMPLEMENTATION 13
4.1 Overview of the Implementation Process 13
4.2 News Article Extraction Using Trafilatura 13
4.3 Text Cleaning and Preprocessing 13
4.4 HTML Tag and URL Removal 13
4.5 Text Normalization 13
4.6 Tokenization Using Hugging Face Tokenizer 13
4.7 Token Counting and Input Length Management 14
4.8 Chunk-Based Text Segmentation 14
4.9 DistilBART Model Loading and Configuration 14
4.10 AI-Based Abstractive Summarization 14
4.11 Multi-Stage Summarization for Long Articles 15
4.12 Summary Length Configuration 15
4.13 Calculation of Summary Statistics 15
4.14 Compression Ratio Calculation 15
4.15 Streamlit Web Application Development 16
4.16 Summary Display and Download Functionality 16

4.17 TextRank Extractive Baseline 16
4.18 Evaluation Methodology 16
CHAPTER 5 – RESULTS AND DISCUSSION 18
5.1 Overview of the Developed Application 18
5.2 User Interface and Application Screenshots 18
5.3 Sample News Article Input 18
5.4 Extracted News Article Output 18
5.5 AI-Generated Summary Output 19
5.6 Summary Statistics and Compression Ratio 19
5.7 Testing of Different Article Lengths 19
5.8 Performance Analysis 20
5.9 Evaluation of Generated Summaries 20
5.10 Observations and Discussion 21
CHAPTER 6 – CONCLUSION AND FUTURE SCOPE 22
6.1 Conclusion 22
6.2 Project Achievements 22
6.3 Limitations of the Proposed System 22
6.4 Future Enhancements 22
6.5 Real-World Applications 23
REFERENCES 24
APPENDICES 25
Appendix A – Source Code 25
Appendix B – Sample News Articles 29
Appendix C – Additional Output Screenshots 30
Appendix D – Testing Results 31

LIST OF FIGURES

No.	Title
Figure 3.1	System architecture of the proposed system
Figure 3.2	Level-0 Data Flow Diagram
Figure 3.3	Use case diagram
Figure 3.4	Flowchart of the proposed system
Figure 5.1	Home screen of the application
Figure 5.2	Sample news URL entered by the user
Figure 5.3	Extracted article output
Figure 5.4	AI-generated summary output
Figure 5.5	Summary statistics and compression ratio
Figure C.1	Additional output screenshots

LIST OF TABLES

No.	Title
Table 2.1	Extractive versus abstractive summarization
Table 2.2	Hardware requirements
Table 2.3	Software requirements
Table 3.1	Functional requirements
Table 3.2	Non-functional requirements
Table 4.1	Summary length settings
Table 4.2	DistilBART generation parameters
Table 5.1	Results for different article lengths
Table 5.2	ROUGE comparison of TextRank and DistilBART
Table 5.3	Factual consistency comparison
Table 5.4	Latency comparison
Table 5.5	Qualitative comparative analysis

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BART	Bidirectional and Auto-Regressive Transformers
CNN	Cable News Network (dataset source)
CPU	Central Processing Unit
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
GPU	Graphics Processing Unit
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
LCS	Longest Common Subsequence
ML	Machine Learning
MNLI	Multi-Genre Natural Language Inference
NLI	Natural Language Inference
NLP	Natural Language Processing
NLTK	Natural Language Toolkit
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
T5	Text-to-Text Transfer Transformer
TF-IDF	Term Frequency – Inverse Document Frequency
URL	Uniform Resource Locator
UI	User Interface

CHAPTER 1 – INTRODUCTION

1.1 Introduction to Natural Language Processing (NLP)

Natural Language Processing is the branch of artificial intelligence concerned with enabling computers to read, interpret and generate human language. Typical NLP tasks include text classification, machine translation, question answering, named-entity recognition and summarization. Early systems relied on hand-written rules and statistical features such as word counts. Since 2017, neural architectures based on the Transformer have replaced most of these methods, because they model long-range relationships between words and can be pretrained on very large text collections and then adapted to specific tasks.

Summarization is among the most demanding of these tasks, because a system must decide what is important, remove what is not, and, in the abstractive case, write fluent new text that remains faithful to the source.

1.2 Background of Automatic Text Summarization

Automatic summarization has been studied since the 1950s, when Luhn proposed ranking sentences by the frequency of significant words. Later work introduced graph-based ranking, with TextRank (Mihalcea and Tarau, 2004) adapting the PageRank algorithm to sentences. Neural sequence-to-sequence models with attention made abstractive summarization practical, and pointer-generator networks (See et al., 2017) reduced the copying and repetition problems of early models. Pretrained encoder–decoder Transformers such as BART (Lewis et al., 2020) and T5 (Raffel et al., 2020) now give the strongest results on news benchmarks such as CNN/DailyMail.

Distillation compresses a large model into a smaller one. Shleifer and Rush (2020) distilled BART into DistilBART variants that keep most of the summarization quality with fewer decoder layers, which makes inference cheaper on ordinary hardware.

1.3 Need for News Summarization

News publishers produce more material each day than any reader can follow. Individual articles often open with the main event, then add quotes, history and reactions. A summary helps in several ways:

- It reduces reading time and lets a reader decide whether to read the full article.
- It supports monitoring tasks, for example analysts tracking many sources at once.
- It improves accessibility for readers who prefer short text or use small screens.
- It can serve as a first step for downstream tools such as alerts, newsletters and search snippets.

A news summary carries a specific risk: a changed name, number or attribution can mislead the reader. A good system must therefore be judged on factual fidelity as well as brevity.

1.4 Problem Statement

Given the URL of an online news article, the system must extract the article text, produce a concise summary whose length the user can control, report how much the text was compressed, and allow the summary to be compared with a classical extractive baseline. The central research question is: **how do sentence selection (TextRank) and neural generation (DistilBART) differ in content coverage, fluency and reliability?**

1.5 Objectives of the Project

1. To build an end-to-end pipeline that extracts a news article from a URL and cleans it for modelling.
2. To implement abstractive summarization with the pretrained DistilBART checkpoint, with chunking for articles longer than the model's input limit.
3. To implement a graph-based extractive baseline using TextRank.
4. To provide short, medium and long summary settings and to compute word counts and compression ratio.
5. To compare the two methods using ROUGE-1, ROUGE-2 and ROUGE-L, sentence-level factual consistency and latency.
6. To generate a timestamped report that the user can save.

1.6 Scope of the Project

The project covers English-language news articles that are publicly accessible through a URL and whose main text Trafilatura can extract. It runs on a CPU and does not require a GPU. The abstractive component uses a pretrained model without further fine-tuning. The evaluation component compares TextRank and DistilBART on a labelled test set with human-written reference summaries.

The following are outside the scope of the present work: multilingual summarization, summarization of multiple articles into one, paywalled pages, video or audio news, and fine-tuning on a new corpus.

1.7 Existing System and Its Limitations

Readers currently rely on three kinds of tools: manual skimming, publisher-written summaries or "highlights", and generic online summarizers. These have the following limitations:

- Manual skimming is slow and inconsistent, and depends on the reader's attention.

- Publisher highlights are written by editors, are not available for every article and cannot be generated on demand.
- Many simple summarizers are purely extractive. They copy sentences, so the result can read as fragmented, contain unresolved references such as “he” or “the company”, and cannot compress below sentence boundaries.
- Basic neural tools often truncate input at the model limit, so long articles lose their later content, and they give the user no statistics or traceable report.
- Few tools let the user compare an extractive and an abstractive result on the same article.

1.8 Proposed System

The proposed system accepts a news URL, retrieves and cleans the article, and summarises it with DistilBART. Long articles are split into token-limited chunks, summarised chunk by chunk, and condensed again in a second stage. The user chooses the summary length. The system calculates original and summary word counts and the compression ratio, and saves a TXT report with the title, author, date, summary, length setting and timestamp.

For comparison, the project adds a TextRank extractive module and an evaluation script that computes ROUGE, factual consistency and latency for both methods on the same articles. Together these give a practical tool and a structured study of the trade-offs between extraction and generation.

1.9 Organization of the Report

- **Chapter 1** introduces the problem, objectives, scope and the proposed system.
- **Chapter 2** reviews summarization techniques, the Transformer, BART and DistilBART, and lists the technologies, hardware and software used.
- **Chapter 3** presents the system analysis and design: architecture, workflow, requirements, modules and diagrams.
- **Chapter 4** describes the methodology and implementation in detail.
- **Chapter 5** presents the application, outputs, the evaluation protocol and the comparison tables.
- **Chapter 6** concludes the report with achievements, limitations, future work and applications.

References and four appendices (source code, sample articles, additional screenshots and testing results) follow Chapter 6.

CHAPTER 2 – LITERATURE REVIEW AND TECHNOLOGY OVERVIEW

2.1 Introduction to Text Summarization Techniques

Summarization methods are usually grouped by how the summary is produced. **Extractive** methods select the most important units, normally sentences, from the source and arrange them as the summary. **Abstractive** methods read the source and generate new sentences that may paraphrase, merge or shorten the original wording. A third group, hybrid methods, first selects content and then rewrites it.

Classical extractive systems score sentences by word frequency, position or similarity to the title. Graph-based methods such as TextRank and LexRank score a sentence by its centrality in a similarity graph. Neural extractive models learn to label sentences as "in" or "out" of the summary. Abstractive systems are almost always neural encoder–decoder models.

2.2 Extractive vs Abstractive Text Summarization

Table 2.1: Extractive versus abstractive summarization

Aspect	Extractive (TextRank)	Abstractive (DistilBART)
Output	Original sentences from the article	Newly generated text
Fluency	Individual sentences are fluent; the whole can feel disconnected	Usually fluent and compact
Compression	Limited by sentence boundaries	Can merge and shorten information
Training data	None required (unsupervised)	Large pretrained model, fine-tuned on summaries
Hallucination risk	No new wording; context can be lost	May alter names, numbers or relations
Compute cost	Light; runs comfortably on CPU	Heavier neural decoding
Traceability	Every sentence maps to the source	Output must be checked against the source

2.3 Introduction to Transformer Architecture

The Transformer (Vaswani et al., 2017) is a neural architecture built around **self-attention**. In self-attention each token computes a weighted combination of all other tokens in the sequence, with the weights learned from queries, keys and values. Because every token can attend directly to every other token, the model captures long-range dependencies without the step-by-step recurrence of earlier RNN models, and training can be parallelised.

A Transformer is made of stacked layers, each with multi-head attention, a feed-forward network, residual connections and layer normalisation. Positional encodings tell the model the order of tokens. The original design is an **encoder–decoder**: the encoder turns the input tokens into contextual representations and the decoder generates the output one token at a

time, attending both to its own previous outputs and to the encoder states. This structure fits summarization naturally, since the article is the input and the summary is the output.

2.4 Overview of BART Model

BART (Lewis et al., 2020) is a denoising sequence-to-sequence model. It is pretrained by corrupting text, for example by masking spans, deleting tokens or shuffling sentences, and training the model to reconstruct the original. Its encoder is bidirectional like BERT, and its decoder is autoregressive like GPT, so it is good both at understanding the input and at generating fluent output. After pretraining, BART is fine-tuned on a downstream task; fine-tuned on CNN/DailyMail it achieved strong summarization results and became a standard baseline. T5 (Raffel et al., 2020) is a related encoder–decoder model that casts every task as text-to-text, and it can be used for the same purpose.

2.5 DistilBART and Its Working Principle

DistilBART is a smaller version of BART obtained by knowledge distillation and layer reduction (Shleifer and Rush, 2020). The checkpoint used in this project, **sshleifer/distilbart-cnn-12-6**, keeps 12 encoder layers and reduces the decoder to 6 layers, and is fine-tuned on CNN/DailyMail. With fewer decoder layers, generation is faster and the model uses less memory than BART-large, which matters because decoding is the slow part of abstractive summarization.

At inference time the working principle is as follows:

1. The tokenizer converts the article into token IDs, truncated or chunked to at most 1,024 tokens.
2. The encoder applies self-attention to build a contextual representation of every token.
3. The decoder generates summary tokens one by one, conditioned on the encoder output and the tokens generated so far.
4. Beam search keeps several candidate continuations (two in this project) and chooses the best-scoring sequence; a no-repeat 3-gram constraint prevents repeated phrases.
5. The generated token IDs are decoded back into text.

2.6 Role of Natural Language Processing in the Project

NLP techniques appear at every stage. Boilerplate removal and text extraction prepare the language data. Cleaning and normalisation remove HTML remnants, URLs and irregular whitespace. Tokenisation maps words and sub-words to the integer IDs the model expects. Sentence segmentation (NLTK) and TF-IDF vectorisation support the TextRank baseline. Finally, the Transformer itself performs language understanding and generation, and an NLI model is used to check whether summary sentences are supported by the source.

2.7 Technologies and Tools Used

- **Python** – the implementation language, chosen for its machine-learning ecosystem.
- **PyTorch** – the deep-learning framework that runs the DistilBART model; inference uses torch.no_grad() on CPU.
- **Hugging Face Transformers** – provides AutoTokenizer and AutoModelForSeq2SeqLM to load the pretrained checkpoint and run generation.
- **DistilBART** – the pretrained abstractive summarization model (sshleifer/distilbart-cnn-12-6).
- **Streamlit** – a Python framework for building the web interface of the application.
- **Trafilatura** – a library that downloads web pages and extracts the main text and metadata, discarding menus, advertisements and comments.
- **HTML and CSS** – used for the styling and layout of the web interface and of the report.
- **VS Code** – the code editor used for development and debugging.

The comparison additionally uses scikit-learn (TF-IDF and cosine similarity), NetworkX (PageRank), NLTK (sentence tokenisation) and the rouge-score package (evaluation).

2.8 Hardware and Software Requirements

Table 2.2: Hardware requirements

Component	Minimum	Recommended
Processor	Quad-core CPU, 2.0 GHz	6-core or better CPU
Memory (RAM)	8 GB	16 GB
Storage	5 GB free (model files and cache)	10 GB free
GPU	Not required (CPU inference)	Optional CUDA GPU for faster decoding
Network	Internet access for article download and first model download	Stable broadband

Table 2.3: Software requirements

Software	Details
Operating system	Windows 10/11, Linux or macOS
Python	3.9 or later
Libraries	torch, transformers, trafilatura, streamlit, nltk, scikit-learn, networkx, rouge-score, pandas, datasets
Model checkpoints	sshleifer/distilbart-cnn-12-6 (summarization); facebook/bart-large-mnli (factual consistency)
Editor / browser	VS Code; any modern web browser

2.9 TextRank for Extractive Summarization

TextRank (Mihalcea and Tarau, 2004) is an unsupervised graph-based ranking algorithm derived from PageRank (Page et al., 1998). For summarization, each sentence of the article is a node. An edge between two sentences carries a weight equal to their similarity. A node's score is computed iteratively: a sentence is important if it is strongly connected to other important sentences. After the scores converge, the top-ranked sentences are selected. In this project similarity is the cosine similarity of TF-IDF vectors, and selected sentences are output in their original order so that the summary follows the article's narrative.

2.10 Evaluation Metrics: ROUGE and Factual Consistency

ROUGE (Lin, 2004) compares a generated summary with one or more human reference summaries. ROUGE-1 counts overlapping unigrams, ROUGE-2 overlapping bigrams, and ROUGE-L the longest common subsequence. Each is reported as precision, recall and F1; this report uses F1. ROUGE measures lexical overlap only. A correct paraphrase can score low, and a summary that copies the right words in the wrong relationship can score high.

Factual consistency asks whether every claim in the summary is supported by the source. Research on abstractive summarization shows that fluent summaries often contain unsupported content (Maynez et al., 2020; Kryscinski et al., 2020). A practical automatic proxy uses an NLI model: each summary sentence is the hypothesis, source passages are the premises, and a sentence counts as supported if some passage entails it with probability at least 0.5. This is a proxy, not ground truth, and ambiguous cases should be checked by human review.

CHAPTER 3 – SYSTEM ANALYSIS AND DESIGN

3.1 System Architecture

The system is organised as a pipeline. A URL enters the extraction module; the cleaned text is tokenised and, if needed, chunked; the text then goes to either the abstractive DistilBART path or the extractive TextRank path. The outputs feed the statistics module, the report module and the evaluation module.

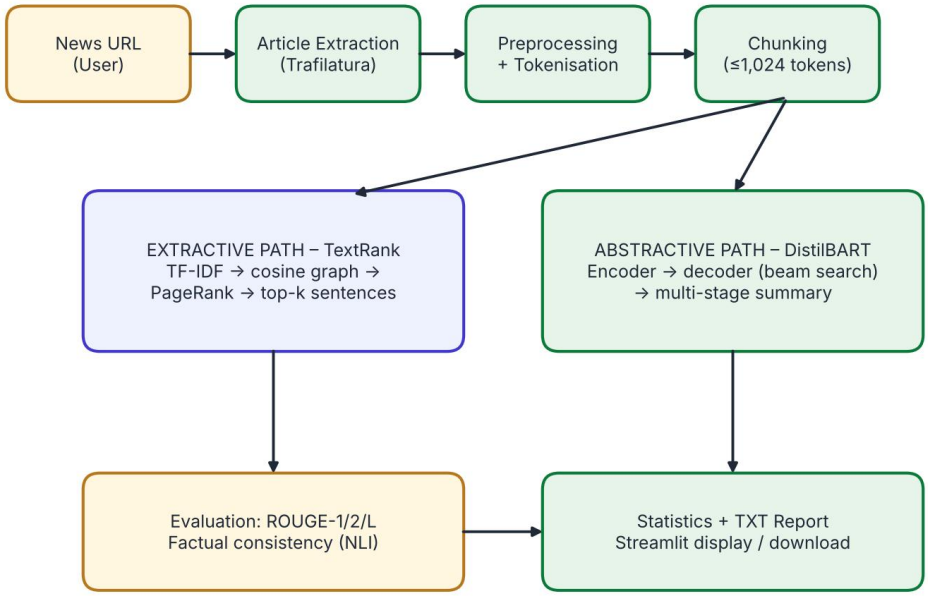


Figure 3.1: System architecture of the proposed system

3.2 Overall Workflow of the Proposed System

1. The user enters the URL of a news article and selects a summary length.
2. Trafilatura downloads the page and extracts the title, author, date and main text.
3. The text is cleaned (HTML tags, URLs, extra whitespace) and normalised.
4. The tokenizer counts tokens. Articles within 1,024 tokens go through in one pass; longer articles are split into chunks.
5. DistilBART generates a summary for each chunk. For long articles, the partial summaries are combined and summarised again.
6. The statistics module computes word counts and the compression ratio.
7. The summary and statistics are displayed and saved to a timestamped TXT report.
8. For evaluation, the same articles are also summarised with TextRank and both outputs are scored against references.

3.3 Functional Requirements

Table 3.1: Functional requirements

ID	Requirement
FR1	Accept a news article URL and reject empty input.
FR2	Extract the title, author, date and body text of the article.
FR3	Clean and normalise the extracted text.
FR4	Count tokens and split articles longer than 1,024 tokens into chunks.
FR5	Generate an abstractive summary with DistilBART.
FR6	Let the user choose a short, medium or long summary.
FR7	Calculate original word count, summary word count and compression ratio.
FR8	Display the summary and save a downloadable report.
FR9	Generate an extractive TextRank summary for comparison.
FR10	Compute ROUGE-1/2/L, factual consistency and latency for both methods on a test set.

3.4 Non-Functional Requirements

Table 3.2: Non-functional requirements

Attribute	Requirement
Performance	Run on a CPU; model loaded once per session; inference with gradient tracking disabled.
Usability	Simple interface; one URL field and one length selector; clear error messages.
Reliability	Handle failed downloads, empty extractions and very long articles without crashing.
Portability	Run on Windows, Linux and macOS with a standard Python environment.
Maintainability	Separate modules for extraction, summarization and reporting.
Transparency	Report the length setting, statistics and timestamp with every summary.

3.5 Module Description

- **News Article URL Input Module** – reads the URL and the summary length from the user and validates that the URL is not empty.
- **Article Extraction Module** – uses Trafilatura to fetch the page and return the title, author, date and main text; the article is also saved to disk.
- **Text Preprocessing Module** – removes HTML tags and URLs, collapses whitespace and normalises the text.
- **Tokenization Module** – uses the DistilBART tokenizer to convert text to token IDs and to count tokens.
- **Text Chunking Module** – splits articles that exceed the input limit into chunks of at most 1,024 tokens.

- **AI Summarization Module** – loads DistilBART, summarises each chunk with beam search and, for long articles, runs a second summarization stage.
- **Summary Statistics Module** – calculates word counts and the compression ratio of the summary.
- **Report Generation and Download Module** – writes the title, author, date, summary, length setting, statistics and timestamp to a TXT file that the user can download.
- **Extractive (TextRank) Module** – added for the comparative study; produces a sentence-selection summary.
- **Evaluation Module** – computes ROUGE, factual consistency and latency for both methods and writes a CSV.

3.6 Data Flow Diagram (DFD)

The level-0 DFD shows the system as a single process that exchanges data with three external entities: the user, the news website and the model hub from which the pretrained weights are downloaded.

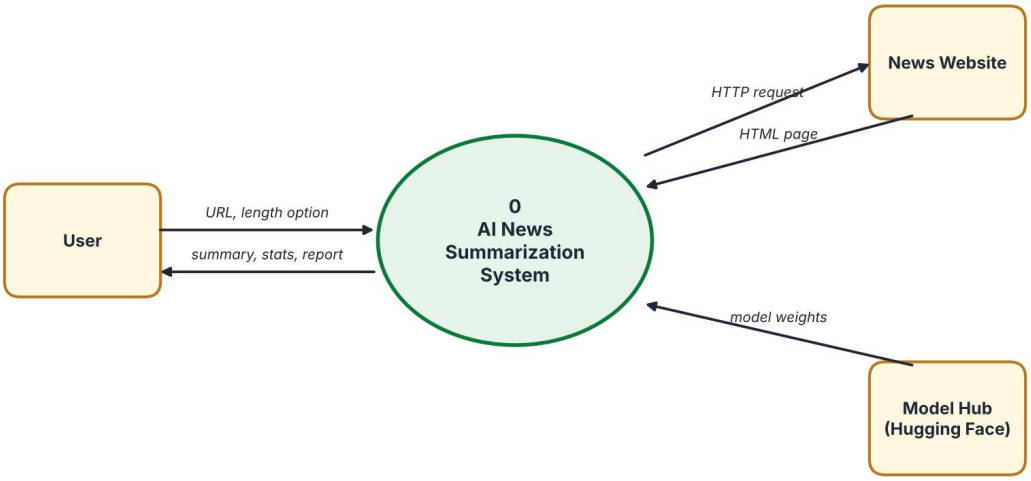


Figure 3.2: Level-0 Data Flow Diagram

3.7 Use Case Diagram

The user enters a URL, selects the length, views the extracted article, generates the summary, views statistics and downloads the report. An evaluator additionally runs the comparison between TextRank and DistilBART.

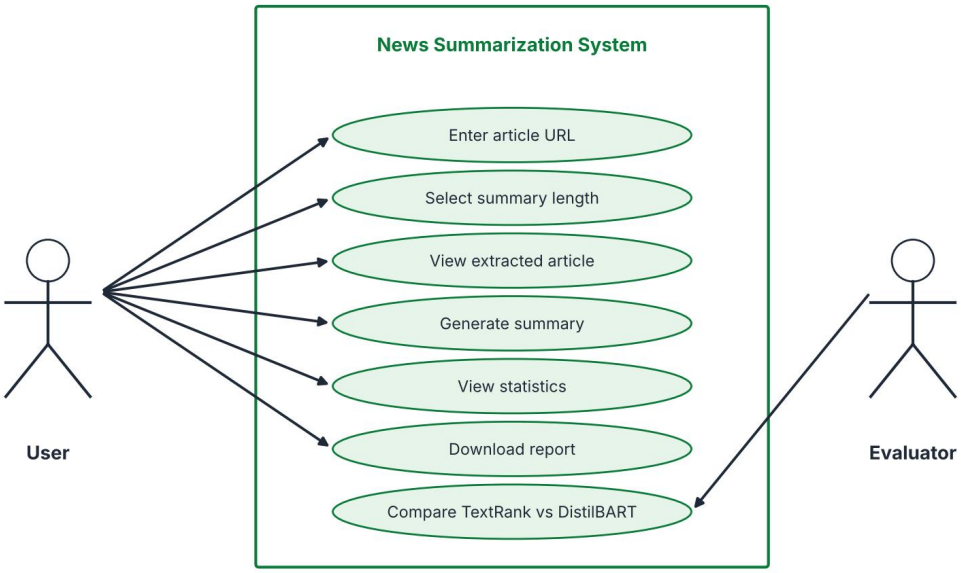


Figure 3.3: Use case diagram

3.8 Flowchart of the Proposed System

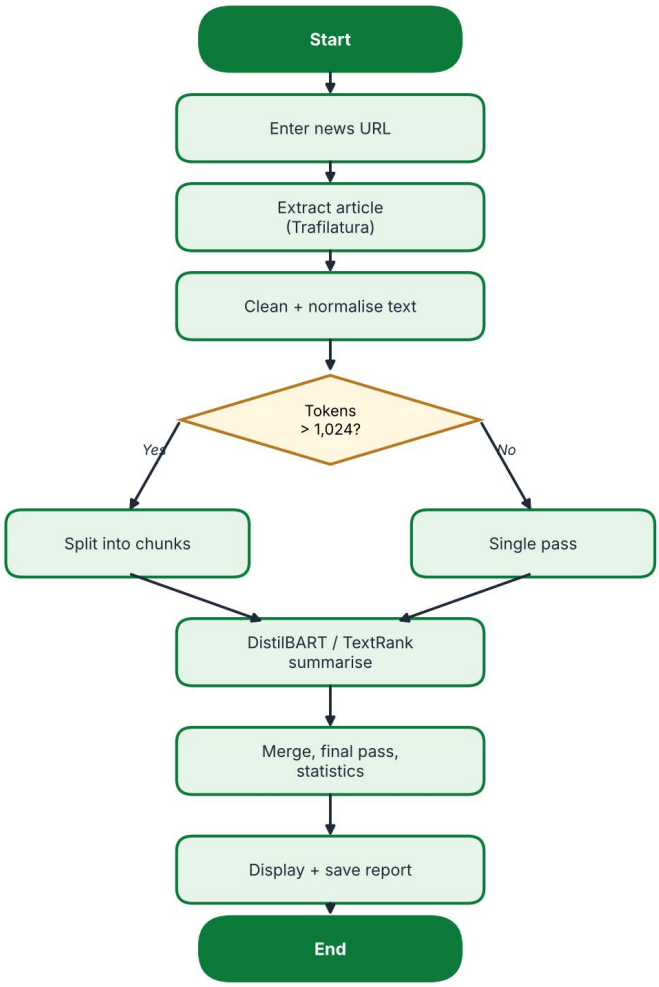


Figure 3.4: Flowchart of the proposed system

3.9 Input and Output Design

Input. A news article URL (text field) and a summary length (short, medium or long).

Output. The extracted article (title, author, date, text); the generated summary; the original and summary word counts; the compression ratio; and a timestamped report saved as outputs/final_summary.txt. The evaluation script outputs a CSV with one row per article and method, containing ROUGE-1, ROUGE-2, ROUGE-L, factual consistency, latency and word counts.

CHAPTER 4 – METHODOLOGY AND IMPLEMENTATION

4.1 Overview of the Implementation Process

The implementation follows the pipeline of Chapter 3 and is organised in separate Python modules: article_extractor.py (retrieval and saving of the article), summarizer.py (model loading, chunking, generation and statistics) and main.py (orchestration, user input and report saving). The comparative study adds textrank_summarizer.py and evaluate.py. The executable path of the main application is: URL → extraction → DistilBART → statistics → TXT report.

4.2 News Article Extraction Using Trafilatura

Trafilatura downloads the HTML of the page and removes navigation menus, advertisements, comment sections and footers, returning the main article text and metadata. The extractor function returns a dictionary containing the title, author, date and article text. The extracted article is saved to disk by save_article() so that the exact input of each summary is preserved. If the download fails or no text is found, an error is raised and displayed to the user instead of passing empty text to the model.

4.3 Text Cleaning and Preprocessing

Extracted text can still contain artefacts that waste model tokens or confuse generation. The cleaning step removes HTML remnants and URLs, replaces repeated whitespace and line breaks with single spaces, and trims the result. The aim is to give the model continuous prose that matches the news text it was fine-tuned on.

4.4 HTML Tag and URL Removal

Regular expressions remove any residual angle-bracket tags and any http/https or www links. URLs are removed because they add many sub-word tokens that carry no meaning for the summary and may appear in the generated output as noise.

4.5 Text Normalization

Normalisation standardises whitespace and punctuation spacing so that sentences are well-formed. Case is preserved, because capitalisation helps the model recognise names and organisations, and because DistilBART's tokenizer is case-sensitive.

4.6 Tokenization Using Hugging Face Tokenizer

The tokenizer loaded with AutoTokenizer.from_pretrained("sshleifer/distilbart-cnn-12-6") uses byte-level byte-pair encoding. It splits text into sub-word tokens, maps them to integer IDs, adds the special start and end tokens, and returns both input_ids and an attention_mask

that marks real tokens versus padding. Sub-word units let the model handle rare words and names by assembling them from known pieces.

4.7 Token Counting and Input Length Management

DistilBART accepts at most 1,024 input tokens. The system counts the tokens of the cleaned article before generation. If the count is within the limit, the article is summarised in a single pass. If it exceeds the limit, the article is passed to the chunking step. Counting tokens, rather than words, is essential because one word can become several tokens.

4.8 Chunk-Based Text Segmentation

A long article is divided into chunks that each fit within the 1,024-token input cap. Splitting at sentence boundaries is preferable to splitting at arbitrary token positions, since it avoids cutting a sentence in half and keeps each chunk readable for the model. Every chunk is summarised independently, so no part of the article is silently dropped by truncation.

4.9 DistilBART Model Loading and Configuration

The model is loaded once through load_model(), which returns the tokenizer and the model. It is moved to the CPU and switched to evaluation mode with model.eval(), which disables dropout. The loaded objects are then passed to generate_summary() for each request so that the heavy loading step is not repeated.

```
MODEL NAME = "sshleifer/distilbart-cnn-12-6"

tokenizer = AutoTokenizer.from pretrained(MODEL NAME)
model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
model.to(torch.device("cpu"))
model.eval()
```

4.10 AI-Based Abstractive Summarization

For each chunk the tokenizer output is passed to model.generate() inside torch.no_grad(), which turns off gradient tracking to save memory and time. Generation uses beam search with two beams, a no-repeat constraint on 3-grams, and early stopping once all beams finish.

Table 4.2: DistilBART generation parameters

Parameter	Value	Purpose
num_beams	2	Keeps two candidate sequences during decoding
no_repeat_ngram_size	3	Prevents any 3-gram from appearing twice
early_stopping	True	Stops when all beams have produced an end token
Input cap	1,024 tokens	Maximum encoder input per chunk
min_length / max_length	Set by summary length	Controls the size of the generated summary

Parameter	Value	Purpose
Device	CPU	No GPU needed

```
with torch.no_grad():
    ids = model.generate(
        input_ids=inputs["input_ids"],
        attention_mask=inputs["attention_mask"],
        max_length=settings["max_length"],
        min_length=settings["min_length"],
        num_beams=2,
        no_repeat_ngram_size=3,
        early_stopping=True
    )
```

4.11 Multi-Stage Summarization for Long Articles

For articles that required chunking, the system uses two stages. In stage one, each chunk is summarised separately. In stage two, the partial summaries are joined and, if the joined text is still long, summarised again to give one coherent final summary. This hierarchical approach lets the system cover the entire article while still respecting the model's input limit. Its main risk is that detail lost in stage one cannot be recovered in stage two, so very long articles may lose minor points.

4.12 Summary Length Configuration

The user selects one of three presets. Each preset sets the minimum and maximum number of generated tokens.

Table 4.1: Summary length settings

Setting	min_length (tokens)	max_length (tokens)
Short	20	80
Medium	40	130
Long	60	180

4.13 Calculation of Summary Statistics

calculate_statistics() counts the words of the original article and of the summary by splitting on whitespace. These counts, together with the compression ratio, the selected length setting and the timestamp, are written to the report.

4.14 Compression Ratio Calculation

The compression ratio expresses how much shorter the summary is than the article:

$$\textit{Compression ratio (\%)} = (\textit{Summary words} \div \textit{Original words}) \times 100$$

For example, a 600-word article summarised in 90 words has a ratio of 15%, meaning the summary keeps 15% of the original length (a reduction of 85%). A lower ratio means stronger compression, but compression alone is not a quality measure, since a very short summary may omit key facts.

4.15 Streamlit Web Application Development

Streamlit provides the web interface for the same pipeline. The interface has a text box for the URL, a selector for short, medium or long summaries, and a button that starts processing. The model is cached with st.cache_resource so that it is loaded once per server session and not on every interaction. A spinner is shown during extraction and generation, and errors from the extractor are displayed as readable messages. HTML and CSS are used to style the layout, headings and the summary card.

4.16 Summary Display and Download Functionality

After generation, the interface shows the article metadata, the summary, the word counts and the compression ratio. A download button (st.download_button) offers the timestamped report as a TXT file. The command-line path writes the same report to outputs/final_summary.txt.

4.17 TextRank Extractive Baseline

The extractive baseline is implemented in textrank_summarizer.py. The steps are:

1. Split the article into sentences with NLTK sent_tokenize.
2. Convert the sentences to TF-IDF vectors with English stop words removed.
3. Compute the pairwise cosine similarity matrix and set its diagonal to zero so that no sentence links to itself.
4. Build a weighted graph from the matrix and run PageRank with NetworkX.
5. Select the top-k sentences (k = 3 by default) and output them in their original order.

As an illustration, running the module on a short seven-sentence budget story with top_k = 2 selected “The budget includes funds for road repairs and public transport.” and “Council members debated the plan for six hours.” This toy test only demonstrates that the module runs; it is not an evaluation result.

TF-IDF is a practical lexical representation. Sentence embeddings could capture semantic similarity better, but they would change the character of the baseline from a lightweight lexical method to a neural one.

4.18 Evaluation Methodology

The evaluation script evaluate.py applies both methods to the same articles and records the following for each article and method:

- **ROUGE-1, ROUGE-2, ROUGE-L (F1)** computed with the rouge-score package with stemming, against the human-written reference summary (the "highlights" field of CNN/DailyMail).
- **Factual consistency** – the share of summary sentences supported by the article. Each summary sentence is paired with source chunks of about 350 tokens and scored by facebook/bart-large-mnli; a sentence is supported if the best entailment probability is at least 0.5.
- **Latency** – wall-clock seconds to produce the summary, excluding model loading, summarised as the median over articles.
- **Lengths** – summary and article word counts, so that overlap scores can be read in light of summary length.

The protocol recommends at least 100 test articles, the same articles for both methods, a fixed random seed or fixed test slice, and a manual check of a sample of sentences where the NLI proxy is uncertain. The results are saved to a CSV file that feeds the tables in Chapter 5.

CHAPTER 5 – RESULTS AND DISCUSSION

5.1 Overview of the Developed Application

The developed application takes a news URL, retrieves the article, produces an abstractive summary in the chosen length, and reports statistics and a downloadable report. A TextRank module and an evaluation script extend the project into a comparative study of the two summarization strategies.

Note: *This chapter separates what has been implemented from what has been measured. Screenshots and the numeric comparison tables are to be filled in from the team's own runs. Cells marked "Pending" must be replaced with values computed by evaluate.py; no figures have been assumed.*

5.2 User Interface and Application Screenshots

[Insert screenshot: home screen with URL field and length selector]

Figure 5.1: Home screen of the application

5.3 Sample News Article Input

The user pastes the URL of a news article and selects a summary length. Appendix B lists the articles used for testing.

[Insert screenshot: URL entered in the input field]

Figure 5.2: Sample news URL entered by the user

5.4 Extracted News Article Output

After the URL is submitted, Trafilatura returns the title, author, date and body text. This output is displayed so that the user can confirm that the correct article was captured before reading the summary.

[Insert screenshot: extracted title, author, date and article text]

Figure 5.3: Extracted article output

5.5 AI-Generated Summary Output

DistilBART generates the summary according to the length preset. The result is shown with the article's metadata.

[Insert screenshot: generated summary]

Figure 5.4: AI-generated summary output

5.6 Summary Statistics and Compression Ratio

The statistics panel shows the original word count, the summary word count, the compression ratio and the timestamp, which are also stored in the report.

[Insert screenshot: statistics panel and download button]

Figure 5.5: Summary statistics and compression ratio

5.7 Testing of Different Article Lengths

Articles of different lengths were chosen to exercise the single-pass path and the chunked, multi-stage path. The table is filled from the test run recorded in Appendix D.

Table 5.1: Results for different article lengths

Article	Words	Tokens	Path	Summary words	Compression (%)
Short article	Pending	Pending	Single pass	Pending	Pending
Medium article	Pending	Pending	Single pass	Pending	Pending

Article	Words	Tokens	Path	Summary words	Compression (%)
Long article (> 1,024 tokens)	Pending	Pending	Chunked, two-stage	Pending	Pending

5.8 Performance Analysis

Performance is measured as the median seconds per article over repeated runs, with the model-loading time excluded, reported together with article token count and device. DistilBART runs on CPU by design, and neural decoding with beam search is expected to take longer than TextRank's graph construction and PageRank. This expectation must be confirmed by measurement.

Table 5.4: Latency comparison

Method	Device	Median seconds / article	Notes
TextRank	CPU	Pending	Excludes NLTK data download
DistilBART	CPU	Pending	Excludes model loading; beam width 2

5.9 Evaluation of Generated Summaries

5.9.1 ROUGE Comparison

Both systems are scored on the same held-out articles against human reference summaries.

Table 5.2: ROUGE comparison of TextRank and DistilBART

Method	ROUGE-1 (F1)	ROUGE-2 (F1)	ROUGE-L (F1)
TextRank	Pending	Pending	Pending
DistilBART	Pending	Pending	Pending

Interpretation caution: extractive systems can gain lexical overlap by copying the source's wording, which does not prove better coverage or accuracy. DistilBART may paraphrase correctly and still receive lower overlap. ROUGE therefore needs to be read together with summary length and the factual measures below.

5.9.2 Factual Consistency

Each summary sentence is labelled as supported, contradicted or not stated by the article. The automatic proxy uses the NLI entailment score; ambiguous cases are adjudicated manually.

Table 5.3: Factual consistency comparison

Method	Sentences evaluated	Supported (%)	Not supported (%)
TextRank	Pending	Pending	Pending

Method	Sentences evaluated	Supported (%)	Not supported (%)
DistilBART	Pending	Pending	Pending

The following should be checked in each generated claim: entities (names, organisations, locations), quantities (dates, counts, percentages), relations (who did what to whom) and qualifiers (alleged, planned, reported, confirmed).

5.10 Observations and Discussion

The two methods optimise different properties. The table below records the expected qualitative behaviour, which the measured results should be used to confirm or revise.

Table 5.5: Qualitative comparative analysis

Dimension	TextRank (extractive)	DistilBART (abstractive)	Evidence status
Latency	Usually lightweight: graph construction and PageRank	Heavier: neural decoding and beam search	Measure (Table 5.4)
Coherence	May feel fragmented when sentences are picked independently	Often more fluent and compact	Qualitative expectation
Compression	Limited by sentence boundaries	Can merge and rewrite information	Measure compression ratio
Hardware	CPU-friendly, sparse vectors and graph operations	Runs on CPU in this project but slower	Measure (Table 5.4)
Factual risk	No new generated claims; context can still be lost	May introduce unsupported details or alter relations	Measure (Table 5.3)

On factuality. Because TextRank copies sentences from the article, it cannot invent new entities or claims. This lowers hallucination risk, but it does not guarantee factual accuracy: selecting a sentence can remove qualifiers, leave a pronoun without its referent, or omit the context that corrects a misleading impression. DistilBART writes new wording and may merge facts, change attribution or overstate a causal link. For this reason the report does not claim that either system is “100% factual”.

On the choice of method. Extractive selection is preferable when traceability is the priority, since every output sentence can be located in the source. Abstractive generation is preferable when readability and compression matter, provided the output is verified against the article.

CHAPTER 6 – CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project built a news summarization system that retrieves an article from a URL, cleans it, and produces an abstractive summary with DistilBART at a user-selected length, with chunk-based handling for long articles, statistics and a saved report. A TextRank baseline and an evaluation framework were added so that extractive and abstractive summarization can be compared on ROUGE, factual consistency and latency. The study shows that the two approaches trade off fluency and compression against traceability, and that overlap metrics must be paired with a factual check before any summary is trusted.

6.2 Project Achievements

- URL-based article extraction with metadata capture using Trafilatura.
- DistilBART abstractive summarization running on CPU.
- Short, medium and long generation settings.
- Chunk-based, multi-stage handling of articles longer than the model input limit.
- Word-count and compression-ratio statistics with a timestamped TXT report.
- A TextRank extractive baseline, tested to run correctly.
- An evaluation script that computes ROUGE-1/2/L, NLI-based factual consistency and latency.

6.3 Limitations of the Proposed System

- The model is used as released; it is not fine-tuned on the target news sources.
- Only English text is supported.
- Extraction can fail on paywalled, script-heavy or unusual page layouts.
- Multi-stage summarization can lose minor details of very long articles.
- DistilBART can produce unsupported or altered claims, and the system does not automatically correct them.
- The NLI-based consistency score is an automatic proxy and can mislabel ambiguous cases.
- Execution on CPU keeps the system accessible but makes abstractive generation slower than the extractive baseline.

6.4 Future Enhancements

- Run the full comparison on a larger labelled test set and add human evaluation of coherence and factuality.
- Add a T5 or BART-large variant and compare it with DistilBART.
- Use sentence embeddings in TextRank and compare them with TF-IDF.

- Build a hybrid pipeline that selects sentences first and rewrites them.
- Display unsupported summary sentences to the user automatically.
- Benchmark latency and memory on CPU and GPU, and explore quantised inference (ONNX, TensorRT).
- Add multilingual models and multi-article summarization.

6.5 Real-World Applications

- News aggregation apps and morning-brief newsletters.
- Media monitoring for businesses, government and research teams.
- Search-engine and social-media snippets.
- Accessibility tools for readers with limited time or screen space.
- Digest generation for legal, financial and academic documents.

REFERENCES

[1] M. Lewis, Y. Liu, N. Goyal, et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," Proc. 58th Annual Meeting of the ACL, 2020.

[2] S. Shleifer and A. M. Rush, "Pre-trained Summarization Distillation," arXiv:2010.13002, 2020.

[3] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," Advances in Neural Information Processing Systems (NeurIPS), 2017.

[4] R. Mihalcea and P. Tarau, "TextRank: Bringing Order into Texts," Proc. EMNLP, 2004.

[5] L. Page, S. Brin, R. Motwani and T. Winograd, "The PageRank Citation Ranking: Bringing Order to the Web," Stanford InfoLab Technical Report, 1998.

[6] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," Text Summarization Branches Out, ACL Workshop, 2004.

[7] C. Raffel, N. Shazeer, A. Roberts, et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," Journal of Machine Learning Research, vol. 21, 2020.

[8] K. M. Hermann, T. Kočiský, E. Grefenstette, et al., "Teaching Machines to Read and Comprehend," NeurIPS, 2015 (source of the CNN/DailyMail dataset).

[9] A. See, P. J. Liu and C. D. Manning, "Get To The Point: Summarization with Pointer-Generator Networks," Proc. ACL, 2017.

[10] J. Maynez, S. Narayan, B. Bohnet and R. McDonald, "On Faithfulness and Factuality in Abstractive Summarization," Proc. ACL, 2020.

[11] W. Kryscinski, B. McCann, C. Xiong and R. Socher, "Evaluating the Factual Consistency of Abstractive Text Summarization," Proc. EMNLP, 2020.

[12] T. Wolf, L. Debut, V. Sanh, et al., "Transformers: State-of-the-Art Natural Language Processing," Proc. EMNLP: System Demonstrations, 2020.

[13] A. Barbaresi, "Trafilatura: A Web Scraping Library and Command-Line Tool for Text Discovery and Extraction," Proc. ACL-IJCNLP: System Demonstrations, 2021.

[14] Hugging Face Transformers documentation, https://huggingface.co/docs/transformers

[15] PyTorch documentation, https://pytorch.org/docs

[16] Streamlit documentation, https://docs.streamlit.io

[17] Trafilatura documentation, https://trafilatura.readthedocs.io

[18] scikit-learn, NetworkX and NLTK documentation, https://scikit-learn.org, https://networkx.org, https://www.nltk.org

APPENDICES

Appendix A – Source Code

The listings below are the core modules. The complete project files (article_extractor.py, summarizer.py, main.py and the Streamlit app) accompany the submission.

A.1 main.py (orchestration)

```
from article_extractor import extract_article, save_article
from summarizer import load_model, generate_summary, calculate_statistics

def main():
    display_header()
    url = input("Enter the News Article URL: ").strip()
    if not url:
        print("Error: News URL cannot be empty.")
        return

    length = get_summary_length()
    try:
        tokenizer, model = load_model()
        article_data = extract_article(url)
        article = article_data["article"]
        save_article(article_data)

        summary = generate_summary(article, tokenizer, model, length)
        stats = calculate_statistics(article, summary)
        report_path = save_summary(article_data, summary, stats, length)
        print("Report Location:", report_path)
    except Exception as error:
        print("APPLICATION ERROR:", error)

if __name__ == "__main__":
    main()
```

A.2 summarizer.py (model loading and generation, core excerpt)

```
MODEL_NAME = "sshleifer/distilbart-cnn-12-6"

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
model = AutoModelForSeq2SeqLM.from_pretrained(MODEL_NAME)
model.to(torch.device("cpu"))
model.eval()

with torch.no_grad():
    ids = model.generate(
        input_ids=inputs["input_ids"],
        attention_mask=inputs["attention_mask"],
        max_length=settings["max_length"],
        min_length=settings["min_length"],
        num_beams=2,
        no_repeat_ngram_size=3,
        early_stopping=True
    )
```

A.3 textrank_summarizer.py (extractive baseline)

```
"""Extractive baseline: TextRank over a TF-IDF cosine-similarity sentence
graph."""
import numpy as np
import networkx as nx
from nltk.tokenize import sent_tokenize
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity


def textrank_summary(text, top_k=3):
    sentences = sent_tokenize(text)
    if len(sentences) <= top_k:
        return " ".join(sentences)

    matrix = TfidfVectorizer(stop_words="english").fit_transform(sentences)
    similarity = cosine_similarity(matrix)
    np.fill_diagonal(similarity, 0.0)

    graph = nx.from_numpy_array(similarity)
    scores = nx.pagerank(graph, weight="weight")
    ranked = sorted(scores, key=scores.get, reverse=True)[:top_k]
    return " ".join(sentences[i] for i in sorted(ranked))
```

A.4 evaluate.py (ROUGE, factual consistency and latency)

```
"""Compare TextRank and DistilBART on CNN/DailyMail.

Metrics: ROUGE-1/2/L F1, sentence-level factual consistency (NLI entailment
against the source article), and median latency per article.

Usage:  python evaluate.py --n 100 --out results.csv
Requires: torch transformers datasets rouge-score nltk networkx scikit-learn
pandas
"""
import argparse, time, statistics as st
import nltk, pandas as pd, torch
from datasets import load_dataset
from nltk.tokenize import sent_tokenize
from rouge_score import rouge_scorer
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer, pipeline
from textrank_summarizer import textrank_summary

nltk.download("punkt", quiet=True); nltk.download("punkt_tab", quiet=True)
SUM_MODEL = "sshleifer/distilbart-cnn-12-6"
NLI_MODEL = "facebook/bart-large-mnli"


def distilbart_summary(text, tok, model, max_len=130, min_len=40):
    inp = tok(text, return_tensors="pt", truncation=True, max_length=1024)
    with torch.no_grad():
        ids = model.generate(**inp, max_length=max_len, min_length=min_len,
                             num_beams=2, no_repeat_ngram_size=3,
early_stopping=True)
    return tok.decode(ids[0], skip_special_tokens=True)
```

```
def chunks(text, tok, limit=350):
    """Split source into ~limit-token pieces so the NLI model sees the whole
article."""
    out, cur, n = [], [], 0
    for s in sent_tokenize(text):
        k = len(tok.tokenize(s))
        if n + k > limit and cur:
            out.append(" ".join(cur)); cur, n = [], 0
        cur.append(s); n += k
    if cur: out.append(" ".join(cur))
    return out


def factual_consistency(summary, article, nli, tok):
    """Fraction of summary sentences entailed (p>=0.5) by at least one source
chunk."""
    src = chunks(article, tok)
    sents = sent_tokenize(summary)
    supported = 0
    for s in sents:
        best = 0.0
        for c in src:
            r = nli({"text": c, "text_pair": s}, top_k=None)
            p = next(x["score"] for x in r if
x["label"].lower().startswith("entail"))
            best = max(best, p)
        supported += best >= 0.5
    return supported / max(len(sents), 1)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--n", type=int, default=100)
    ap.add_argument("--out", default="results.csv")
    a = ap.parse_args()

    data = load_dataset("cnn_dailymail", "3.0.0", split=f"test[:{a.n}]")
    tok = AutoTokenizer.from_pretrained(SUM_MODEL)
    model = AutoModelForSeq2SeqLM.from_pretrained(SUM_MODEL).eval()
    nli_tok = AutoTokenizer.from_pretrained(NLI_MODEL)
    nli = pipeline("text-classification", model=NLI_MODEL, tokenizer=nli_tok,
device=-1)
    scorer = rouge_scorer.RougeScorer(["rouge1", "rouge2", "rougeL"],
use_stemmer=True)

    rows = []
    for i, ex in enumerate(data):
        art, ref = ex["article"], ex["highlights"]
        for name, fn in (("TextRank", lambda t: textrank_summary(t, 3)),
                         ("DistilBART", lambda t: distilbart_summary(t, tok,
model))):
            t0 = time.perf_counter(); summ = fn(art); dt = time.perf_counter() -
t0
            r = scorer.score(ref, summ)
            rows.append(dict(id=i, method=name, rouge1=r["rouge1"].fmeasure,
                             rouge2=r["rouge2"].fmeasure,
rougeL=r["rougeL"].fmeasure,
                             factual=factual_consistency(summ, art, nli, nli_tok),
```

```
                                seconds=dt, summary_words=len(summ.split()),
                                article_words=len(art.split())))
            print(f"{i+1}/{a.n}", end="\r")

    df = pd.DataFrame(rows); df.to_csv(a.out, index=False)
    print(df.groupby("method")[["rouge1", "rouge2", "rougeL",
"factual"]].mean().round(4))
    print(df.groupby("method")["seconds"].median().round(3))

if __name__ == "__main__":
    main()
```

Appendix B – Sample News Articles

List the articles used for testing. For each, record the source, URL, publication date and word count.

Table B.1: Sample news articles

No.	Source	Headline	URL	Date	Words
1					
2					
3					
4					
5					

Appendix C – Additional Output Screenshots

[Insert screenshot: additional outputs (long article, error handling, downloaded report)]

Figure C.1: Additional output screenshots

Appendix D – Testing Results

Record the outputs of evaluate.py here (the per-article CSV or its summary), the machine used, and the date of the run.

Table D.1: Test configuration

Item	Value
Test set	CNN/DailyMail 3.0.0, test split, first N articles (N = ____)
Machine / device	
Date of run	
Command	python evaluate.py --n ___ --out results.csv

Table D.2: Summary of evaluation run

Method	ROUGE-1	ROUGE-2	ROUGE-L	Factual (%)	Median s
TextRank					
DistilBART					